

Homework 8

The primary purpose of this assignment is to explore mutation and imperative updates in programming languages. We explore this language feature together with the related notion of parameter passing modes. Concretely, we extend JAKARTAScript with mutable variables and parameter passing modes. Parameters are always passed by value in JavaScript, so the parameter passing modes in JAKARTAScript are an extension beyond JavaScript to illustrate a language design decision. We will update our type checker and interpreter from Homework 7 and see that mutation forces us to do a rather global refactoring of our interpreter.

Try to make your code as concise and clear as possible. Challenge yourself to find the most crisp, concise way of expressing the intended computation. This may mean using ways of expressing computation currently unfamiliar to you.

Problem 1 Parameter Passing Modes (12 Points)

This problem is meant as a warm-up exercise for Problem 2 where you will implement a version of JAKARTAScript that supports different parameter passing modes.

Consider the typed variant of JAKARTAScript that supports the different parameter passing modes we discussed. For each of the following programs, say whether the program is well-typed. If not, **provide a brief explanation of the type error**. If yes, compute the value that the program evaluates to and **provide a brief explanation why you obtain that specific value**.

(a)

```
1 const y = 3;  
2 const f = (let x: Num) => (x = x + 1, x + y);  
3 f(y) + y
```

(b)

```
1 let x = 3;  
2 const f = (name x: Num) => x + x;  
3 f(x = x + 1) + x
```

(c)

```
1 let y = 3;  
2 const f = (let x: Num) => (x = x + 1, x + y);  
3 f(y * 2) + y
```

(d)

```
1 let x = 3;  
2 const f = (ref y: Num) => (y = y + 1, y + x);  
3 f(x) + x
```

(e)

```

1  let y = 3;
2  const f = (ref x: Num) => (x = x + 1, x + y);
3  f(y * 2) + y

```

Problem 2 JAKARTAScript Interpreter with State (28 Points)

At this point, we are used to extending our interpreter implementation by updating our type checker `typeInfer` and our interpreter `eval`. The syntax with the new extensions highlighted is shown in Figure 1.

Mutation. In this assignment, we add mutable variables declared as follows:

$$\mathbf{let} \ x = e_d; e_b$$

and then include an assignment expression:

$$e_1 = e_2$$

that writes the value of e_2 to a location named by expression e_1 .

Parameter Passing Modes. In this assignment, we can annotate function parameters with **const**, **let**, **name**, or **ref** to specify a parameter passing mode. The annotation **let** says that the parameter should be pass-by-value with an allocation for a new mutable parameter variable initialized to the argument value. The **name** and **ref** annotations specify pass-by-name and pass-by-reference, respectively. In Homework 7, all parameters were pass-by-value with an immutable variable. This mode is now captured by a **const** annotation. These “pass-by” terms are defined by their respective `EvalCall` rules in Figure ??.

Addresses a and dereference operations $* a$ are included in program expressions e because they arise during evaluation. However, there is no way to explicitly write these expressions in the source program. Addresses and dereference expressions are examples of enrichments of program expressions as an intermediate form solely for evaluation.

In Figure 2, we show the updated and new AST nodes. Note that `Deref` is a `Uop` and `Assign` a `Bop`.

Type Checking. The inference rules defining the typing relation are given in Figures 3 and 4. Similar to before, we implement type inference with the function

```
def typeInfer(env: Map[String,(Mut,Typ)], e: Expr): Typ
```

that you need to complete. Note that the type environment maps a variable name to a pair of a mutability (either `MConst` or `MLet`) and a type. A template for the `Function` case for `typeInfer` is provided that you may use if you wish.

In the implementation of our type checker we now distinguish two different kinds of type errors. Just like in Homework 7, we have regular static type errors for those situations where we detect a type mismatch. The second kind of type error is used to signal the cases where we expect an assignable expression (i.e., a mutable variable) but find a non-assignable expression. To signal this second kind of type error, we will use a Scala exception

$n \in \text{Num}$	numbers (double)
$s \in \text{Str}$	strings
$a \in \text{Addr}$	addresses
$b \in \text{Bool} ::= \text{true} \mid \text{false}$	Booleans
$x \in \text{Var}$	variables
$\tau \in \text{Typ} ::= \text{Bool} \mid \text{Num} \mid \text{String} \mid \text{Undefined} \mid$ $\text{mode } \tau \Rightarrow \tau_0$	types
$v \in \text{Val} ::= \text{undefined} \mid n \mid b \mid s \mid a \mid$ $\text{function } p(\text{mode } x:\tau) t e$	values
$e \in \text{Expr} ::= x \mid v \mid uop \ e \mid e_1 \ bop \ e_2 \mid e_1 \ ? \ e_2 : e_3 \mid$ $\text{console.log}(e) \mid e_1(e) \mid$ $\text{mut } x = e_1; e_2$	expressions
$uop \in \text{Uop} ::= - \mid ! \mid *$	unary operators
$bop \in \text{Bop} ::= + \mid - \mid * \mid / \mid == \mid != \mid < \mid > \mid$ $<= \mid >= \mid \&\& \mid \mid , \mid =$	binary operators
$p ::= x \mid \epsilon$	function names
$t ::= :\tau \mid \epsilon$	return types
$\text{mut} \in \text{Mut} ::= \text{const} \mid \text{let}$	mutability
$\text{mode} \in \text{PMode} ::= \text{const} \mid \text{let} \mid \text{ref} \mid \text{name}$	passing mode
$M \in \text{Mem} = \text{Addr} \rightarrow \text{Val}$	memories

Figure 1: Abstract syntax of JAKARTA SCRIPT

```
class LocTypeError(e: Expr) extends JsException
```

where e is the non-assignable expression. We also provide a helper function `locerr` to simplify throwing this exception.

For example, in the following program we declare a **const** variable `x`, which we then try to reassign:

```
1 const x = 3;
2 x = 2
```

The function `typeInfer` should reject this program throwing a `LocTypeError` with the expression `x` in the left side of the assignment on line 2 as argument.

Evaluation. We also update `subst` and `eval` from Homework 7. The required modifications to `subst` are minimal and have already been provided for you.

A big-step operational semantics is given in Figures 5 and ???. The big-step judgment form is now as follows:

$$\langle M, e \rangle \Downarrow \langle M', v \rangle$$

```

/** Mutabilities */
enum Mut:
  case MConst, MLet // <~ const, let

/** Parameter Passing Modes */
enum PMode:
  case PConst, PName, PLet, PRef // <~ const, name, let, ref

/** Binary Operators */
enum Bop:
  ...
  case Assign // <~ =

/** Unary Operators */
enum Uop:
  ...
  case Deref // <~ *

/** Expressions */
type Param = (String, (PMode, Typ))
enum Expr:
  ...
  case Decl(mut: Mut, x: String, ed: Expr, eb: Expr) // <~ mut x = ed; eb

  case Function(p: Option[String], x: Param, t: Option[Typ], e: Expr)
    // <~ function p(mode x:  $\tau$ ) t e

  /** Addresses */
  case Addr private[ast] (addr: Int) // <~ a

/** Types */
enum Typ:
  ...
  case TFunction(t: (PMode, Typ), tret: Typ) // <~ mode  $\tau \Rightarrow \tau_{ret}$ 

```

Figure 2: The abstract syntax of JAKARTA SCRIPT as represented in Scala. After each **case**, we show the corresponding JavaScript expression represented by that case.

that says informally, “In input memory M , expression e evaluates to the output memory M' and value v .” The memory M is a map from addresses a to values. The presence of a memory M that gets updated during evaluation is the hallmark of imperative computation.

- The eval function now has the following signature

```
def eval(m: Mem, e: Expr): (Mem, Val)
```

corresponding to the updated operational semantics. This function needs to be completed.

Note that the change in the judgment form necessitates updating all rules, even those that do not involve imperative features as in Figure 5. Some rules require allocating fresh addresses. For example, EVALLETDECL specifies allocating a new address a and extending the memory, mapping a to the value v_d of the defining expression e_d . The address a is stated to be fresh by the constraint that $a \notin \text{dom}(M_d)$. Then M_d is updated to M' by storing v_d at address a in M_d . In the implementation, you call `md.alloc(vd)` to get a pair `(mp, a)`

consisting of a fresh address `a` and an updated memory state `mp` with the location of address `a` initialized to value `vd` in `md`, thus achieving both of the above steps in just one step.

You might notice that in our operational semantics, the memory M only grows and never shrinks during the course of evaluation. Our interpreter only ever allocates memory and never deallocates! This choice is fine in a mathematical model and for this assignment, but a production run-time system must somehow enable collecting garbage—allocated memory locations that are no longer used by the running program. Collecting garbage may be done manually by the programmer (as in C and C++) or automatically by a conservative garbage collector (as in JavaScript, Scala, Java, C#, and Python).

You might also notice that we have a single memory instead of a *stack of activation records* for local variables and a *heap* for objects as discussed in Computer Systems Organization. Our interpreter instead simply allocates memory for local variables when they are encountered (e.g., `EVALLETDECL`). It never deallocates, even though we know that with local variables, those memory locations become inaccessible by the program once the body of a declaration, respectively, function has been evaluation. The key observation is that the traditional stack is not essential for local variables but rather is an optimization for automatic deallocation based on block statements, respectively, function call-and-return.

$$\begin{array}{c}
\overline{\Gamma \vdash b : \mathbf{Bool}} \text{ TYPEBOOL} \quad \overline{\Gamma \vdash n : \mathbf{Num}} \text{ TYPENUM} \quad \overline{\Gamma \vdash s : \mathbf{String}} \text{ TYPESTR} \\
\\
\overline{\Gamma \vdash \mathbf{undefined} : \mathbf{Undefined}} \text{ TYPEUNDEFINED} \\
\\
\frac{\Gamma \vdash e : \mathbf{Num}}{\Gamma \vdash -e : \mathbf{Num}} \text{ TYPEUMINUS} \quad \frac{\Gamma \vdash e : \mathbf{Bool}}{\Gamma \vdash !e : \mathbf{Bool}} \text{ TYPENOT} \\
\\
\frac{\Gamma \vdash e_1 : \mathbf{Bool} \quad \Gamma \vdash e_2 : \mathbf{Bool} \quad bop \in \{\&\&, ||\}}{\Gamma \vdash e_1 bop e_2 : \mathbf{Bool}} \text{ TYPEANDOR} \\
\\
\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1, e_2 : \tau_2} \text{ TYPESEQ} \quad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathbf{console.log}(e) : \mathbf{Undefined}} \text{ TYPEPRINT} \\
\\
\frac{\Gamma \vdash e_1 : \mathbf{String} \quad \Gamma \vdash e_2 : \mathbf{String}}{\Gamma \vdash e_1 + e_2 : \mathbf{String}} \text{ TYPEPLUSSTR} \\
\\
\frac{\Gamma \vdash e_1 : \mathbf{Num} \quad \Gamma \vdash e_2 : \mathbf{Num} \quad bop \in \{+, *, /, -\}}{\Gamma \vdash e_1 bop e_2 : \mathbf{Num}} \text{ TYPEARITH} \\
\\
\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \quad \tau \in \{\mathbf{Num}, \mathbf{String}\} \quad bop \in \{>, >=, <, <=\}}{\Gamma \vdash e_1 bop e_2 : \mathbf{Bool}} \text{ TYPEINEQUAL} \\
\\
\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \quad \tau \text{ has no function types} \quad bop \in \{===, !==\}}{\Gamma \vdash e_1 bop e_2 : \mathbf{Bool}} \text{ TYPEEQUAL} \\
\\
\frac{\Gamma \vdash e_1 : \mathbf{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash e_1 ? e_2 : e_3 : \tau} \text{ TYPEIF}
\end{array}$$

Figure 3: Type checking rules for non-imperative primitives of JAKARTASCRIPT (no changes compared to Homework 7)

$$\begin{array}{c}
\frac{\Gamma \vdash e_d : \tau_d \quad \Gamma' = \Gamma[x \mapsto (mut, \tau_d)] \quad \Gamma' \vdash e_b : \tau_b}{\Gamma \vdash mut\ x = e_d; e_b : \tau_b} \text{TYPEDECL} \\
\\
\frac{x \in \text{dom}(\Gamma) \quad \Gamma(x) = (mut, \tau)}{\Gamma \vdash x : \tau} \text{TYPEVAR} \\
\\
\frac{\Gamma(x) = (\mathbf{let}, \tau) \quad \Gamma \vdash e : \tau}{\Gamma \vdash x = e : \tau} \text{TYPEASSIGNVAR} \\
\\
\frac{\Gamma \vdash e : mode_1 \tau_1 \Rightarrow \tau \quad mode \neq \mathbf{ref} \text{ or } e_1 \in Var \text{ and } \Gamma(e_1) = (\mathbf{let}, \tau_1)}{\Gamma \vdash e(e_1) : \tau} \text{TYPECALL} \\
\\
\frac{\Gamma' = \Gamma[x_1 \mapsto (mut(mode_1), \tau_1)] \quad \Gamma' \vdash e : \tau \quad \tau' = mode_1 \tau_1 \Rightarrow \tau}{\Gamma \vdash (mode_1\ x_1 : \tau_1) \Rightarrow e : \tau'} \text{TYPEFUNANN} \\
\\
\frac{\Gamma' = \Gamma[x \mapsto \tau'][x_1 \mapsto (mut(mode_1), \tau_1)] \quad \Gamma' \vdash e : \tau \quad \tau' = mode_1 \tau_1 \Rightarrow \tau}{\Gamma \vdash \mathbf{function}\ x(mode_1\ x_1 : \tau_1) : \tau\ e : \tau'} \text{TYPEFUNREC} \\
\\
mut(\mathbf{const}) = mut(\mathbf{name}) = \mathbf{const} \\
\\
mut(\mathbf{let}) = mut(\mathbf{ref}) = \mathbf{let}
\end{array}$$

Figure 4: Type checking rules for imperative primitives and functions with parameter passing modes.

$$\begin{array}{c}
\frac{}{\langle M, v \rangle \Downarrow \langle M, v \rangle} \text{EVALVAL} \quad \frac{\langle M, e \rangle \Downarrow \langle M', n \rangle}{\langle M, -e \rangle \Downarrow \langle M', -n \rangle} \text{EVALUMINUS} \quad \frac{\langle M, e \rangle \Downarrow \langle M', b \rangle}{\langle M, !e \rangle \Downarrow \langle M', !b \rangle} \text{EVALNOT} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{true} \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, v_2 \rangle}{\langle M, e_1 \ \&\& \ e_2 \rangle \Downarrow \langle M_2, v_2 \rangle} \text{EVALANDTRUE} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{false} \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, v_2 \rangle}{\langle M, e_1 \ || \ e_2 \rangle \Downarrow \langle M_2, v_2 \rangle} \text{EVALORFALSE} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{false} \rangle}{\langle M, e_1 \ \&\& \ e_2 \rangle \Downarrow \langle M_1, \mathbf{false} \rangle} \text{EVALANDFALSE} \quad \frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{true} \rangle}{\langle M, e_1 \ || \ e_2 \rangle \Downarrow \langle M_1, \mathbf{true} \rangle} \text{EVALORTTRUE} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, v_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, v_2 \rangle}{\langle M, e_1 \ , \ e_2 \rangle \Downarrow \langle M_2, v_2 \rangle} \text{EVALSEQ} \\
\\
\frac{\langle M, e \rangle \Downarrow \langle M', v \rangle \quad v \text{ printed}}{\langle M, \mathbf{console.log}(e) \rangle \Downarrow \langle M', \mathbf{undefined} \rangle} \text{EVALPRINT} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, n_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, n_2 \rangle \quad n = n_1 + n_2}{\langle M, e_1 + e_2 \rangle \Downarrow \langle M_2, n \rangle} \text{EVALPLUSNUM} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, s_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, s_2 \rangle \quad s = s_1 + s_2}{\langle M, e_1 + e_2 \rangle \Downarrow \langle M_2, s \rangle} \text{EVALPLUSSTR} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, n_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, n_2 \rangle \quad n = n_1 \text{ bop } n_2 \quad \text{bop} \in \{*, /, -\}}{\langle M, e_1 \text{ bop } e_2 \rangle \Downarrow \langle M_2, n \rangle} \text{EVALARITH} \\
\\
\frac{\langle M, e_d \rangle \Downarrow \langle M_d, v_d \rangle \quad \langle M_d, e_b[v_d/x] \rangle \Downarrow \langle M', v_b \rangle}{\langle M, \mathbf{const } x = e_d; e_b \rangle \Downarrow \langle M', v_b \rangle} \text{EVALCONSTDECL} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, n_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, n_2 \rangle \quad b = n_1 \text{ bop } n_2 \quad \text{bop} \in \{>, >=, <, <=\}}{\langle M, e_1 \text{ bop } e_2 \rangle \Downarrow \langle M_2, b \rangle} \text{EVALINEQUALNUM} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, s_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, s_2 \rangle \quad b = s_1 \text{ bop } s_2 \quad \text{bop} \in \{>, >=, <, <=\}}{\langle M, e_1 \text{ bop } e_2 \rangle \Downarrow \langle M_2, b \rangle} \text{EVALINEQUALSTR} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, v_1 \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, v_2 \rangle \quad b = (v_1 \text{ bop } v_2) \quad \text{bop} \in \{===, !==\}}{\langle M, e_1 \text{ bop } e_2 \rangle \Downarrow \langle M_2, b \rangle} \text{EVALEQUAL} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{true} \rangle \quad \langle M_1, e_2 \rangle \Downarrow \langle M_2, v_2 \rangle}{\langle M, e_1 \ ? \ e_2 : e_3 \rangle \Downarrow \langle M_2, v_2 \rangle} \text{EVALIFTHEN} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M_1, \mathbf{false} \rangle \quad \langle M_1, e_3 \rangle \Downarrow \langle M_3, v_3 \rangle}{\langle M, e_1 \ ? \ e_2 : e_3 \rangle \Downarrow \langle M_3, v_3 \rangle} \text{EVALIFELSE}
\end{array}$$

Figure 5: Big-step operational semantics of non-imperative primitives of JAKARTAScript. The only change compared to Homework 7 is the threading of the memory.

$$\begin{array}{c}
\frac{\langle M, e \rangle \Downarrow \langle M', v \rangle \quad a \in \text{dom}(M')}{\langle M, *a = e \rangle \Downarrow \langle M'[a \mapsto v], v \rangle} \text{ EVALASSIGNVAR} \\
\\
\frac{a \in \text{dom}(M)}{\langle M, *a \rangle \Downarrow \langle M, M(a) \rangle} \text{ EVALDEREFVAR} \\
\\
\frac{\langle M, e_d \rangle \Downarrow \langle M_d, v_d \rangle \quad a \notin \text{dom}(M_d) \quad M' = M_d[a \mapsto v_d] \quad \langle M', e_b[*a/x] \rangle \Downarrow \langle M'', v_b \rangle}{\langle M, \text{let } x = v_d; e_b \rangle \Downarrow \langle M'', v_b \rangle} \text{ EVALLETDECL} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = (\text{const } x : \tau_2) \Rightarrow e \quad \langle M', e_2 \rangle \Downarrow \langle M'', v_2 \rangle \quad e' = e[v_2/x] \quad \langle M'', e' \rangle \Downarrow \langle M''', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M''', v \rangle} \text{ EVALCALLVAL} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = \text{function } x_1(\text{const } x_2 : \tau_2) : \tau \ e \quad \langle M', e_2 \rangle \Downarrow \langle M'', v_2 \rangle \quad e' = e[v_1/x_1][v_2/x_2] \quad \langle M'', e' \rangle \Downarrow \langle M''', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M''', v \rangle} \text{ EVALCALLVALREC} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = (\text{let } x : \tau_2) \Rightarrow e \quad \langle M', e_2 \rangle \Downarrow \langle M'', v_2 \rangle \quad a \notin \text{dom}(M'') \quad e' = e[*a/x] \quad \langle M''[a \mapsto v_2], e' \rangle \Downarrow \langle M''', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M''', v \rangle} \text{ EVALCALLVAR} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = \text{function } x_1(\text{let } x_2 : \tau_2) : \tau \ e \quad \langle M', e_2 \rangle \Downarrow \langle M'', v_2 \rangle \quad a \notin \text{dom}(M'') \quad e' = e[v_1/x_1][*a/x_2] \quad \langle M''[a \mapsto v_2], e' \rangle \Downarrow \langle M''', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M''', v \rangle} \text{ EVALCALLVARREC} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = (\text{name } x : \tau_2) \Rightarrow e \quad e' = e[e_2/x] \quad \langle M', e' \rangle \Downarrow \langle M'', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M'', v \rangle} \text{ EVALCALLNAME} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = \text{function } x_1(\text{name } x_2 : \tau_2) : \tau \ e \quad e' = e[v_1/x_1][e_2/x_2] \quad \langle M', e' \rangle \Downarrow \langle M'', v \rangle}{\langle M, e_1(e_2) \rangle \Downarrow \langle M'', v \rangle} \text{ EVALCALLNAMEREC} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = (\text{ref } x : \tau_2) \Rightarrow e \quad e' = e[*a/x] \quad \langle M', e' \rangle \Downarrow \langle M'', v \rangle}{\langle M, e_1(*a) \rangle \Downarrow \langle M'', v \rangle} \text{ EVALCALLREF} \\
\\
\frac{\langle M, e_1 \rangle \Downarrow \langle M', v_1 \rangle \quad v_1 = \text{function } x_1(\text{ref } x_2 : \tau_2) : \tau \ e \quad e' = e[v_1/x_1][*a/x_2] \quad \langle M', e' \rangle \Downarrow \langle M'', v \rangle}{\langle M, e_1(*a) \rangle \Downarrow \langle M'', v \rangle} \text{ EVALCALLREFREC}
\end{array}$$

Figure 6: New inference rules that define the big-step semantics of function call expressions for the different parameter passing modes